Reviewer Pack — Minimal Order of Brauer Groups and Resolution Proof Width Co…

Entry 54e1bf78dfd1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 15:15:34 UTC

Minimal Order of Brauer Groups and Resolution Proof Width Correlation

Entry ID: `54e1bf78dfd1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 15:15:34 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Brauer Group Theory) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> For every Boolean satisfiability instance φ with n variables, the minimal order of the Brauer group of the algebraic closure of the polynomial ring R[φ] in characteristic 2 is linearly correlated with its resolution proof width w(φ), such that O(√n) ≤ min_order(Brauer_group(R[φ])) ≤ √n * w(φ).

Rationale (proposer's reasoning):

> The minimal order of the Brauer group can capture non-trivial algebraic properties of the underlying Boolean function, which may be related to the complexity of finding a resolution proof. This conjecture aims to bridge the gap between representation theory and computational complexity by linking algebraic invariants with proof complexity.

Taxonomy category: `BrauerGroup_ResolutionProofWidth` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1990a22a8197d763`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40, the correlation coefficient between min_order(Brauer_group(R[φ])) and √n * w(φ) from 30 random seeds exceeds 0.8, with no seed producing a correlation coefficient below 0.7.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_instance(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_brauer_group_order(instance):
        # Placeholder function to simulate Brauer group order computation
        # This is a dummy implementation and should be replaced with actual logic
        return len(instance)

    def resolution_proof_width(instance):
        # Placeholder function to simulate resolution proof width computation
        # This is a dummy implementation and should be replaced with actual logic
        return len(instance) ** 0.5

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        instance = generate_boolean_instance(n)
        min_order = compute_brauer_group_order(instance)
        w_phi = resolution_proof_width(instance)
        correlation_value = (min_order - math.sqrt(n)) / (math.sqrt(n) * w_phi)
        results.append(correlation_value)

    mean_metric_value = sum(results) / len(results)
    conjecture_holds = all(0.7 <= corr >= 0.8 for corr in results)
    counterexample = "" if conjecture_holds else "Correlation out of bounds"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": mean_metric_value,
        "instances_tested": len(results),
```

```
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes if

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean_metric_value = sum(results) / len(results)
    support_fraction = sum(1 for r in results if 0.7 <= r >= 0.8) / len(results)

    if all(0.7 <= r >= 0.8 for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(r < 0.7 or r > 0.8 for r in results):
        first_failing_seed = seeds[next(i for i, r in enumerate(results) if not (0.7 <= r >= 0.8))
        print(f"RESULT: FALSIFIED counterexample='Correlation out of bounds' first_failing_seed={f
    else:
        print("RESULT: INCONCLUSIVE Reason: No valid correlation values found")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete its intended calculations to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14259
2	preregistration	ollama_remote	glm4:latest	0	0	9290
3	novelty	ollama_remote	glm4:latest	0	0	10322
4	novelty	ollama_remote	glm4:latest	0	0	8521
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18698
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9585
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14776
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8237
9	judge	ollama_remote	glm4:latest	0	0	26568

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 120256 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/54e1bf78dfd1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/54e1bf78dfd1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/54e1bf78dfd1.tar.gz` (if generated)